

Re: Why a push campaign cannot be created via the API

Teamder · 23 August 2026 · verified against the deployed API

Both corrections accepted, and one of them was my error rather than an imprecision.

Correction 1 — you are right, and here is what misled me

My report said " `POST /campaigns` accepts an `appId` field but never populates it". That sentence is wrong, and I should not have written it: I never tested the create path with an `appId`. What I actually did was `PUT` a campaign with an `appId` and watch it come back `null`, then generalise from update to create. Later in the same session I created a campaign *with* `appId` and it persisted — which should have made me go back and correct the claim, and did not.

So the write path was never broken. The read path was, exactly as my own point 3 said. Two different statements in one report, one of them unfounded.

Correction 2 — accepted, and it is worse than I described

`if (!appId) return` at the top of `debugAssertPushAppId` means the null case — the only case that occurred — produced no signal at all. I wrote "it only logs, never throws", which still credits it with a log line it never emitted.

The `sendTestPush` asymmetry is the part I would not have found: a test send refusing campaigns it could deliver, while the real send accepted ones it could not. That is the same shape as the other defects in this integration — the honest error existed, just not on the path anyone hits.

Your point 1 — you are right to push back, and our workspace proves it

I checked as soon as `apps:read` landed:

```
GET /apps
418d3496...  web      Teamder Landing page  pushEnabled=None
61169e4e...  ios      Teamder ios app       pushEnabled=None
582c204f...  android  Teamder android app   pushEnabled=True
```

`apps.find(a => a.platform === 'ios' || a.platform === 'android')` returns the **iOS** app here — the one with no push credentials configured. My suggested default would not have halved

our reach, it would have selected the single app that cannot deliver anything, and reported success. Resolve-when-unambiguous, refuse-when-zero, require-a-name-when-many is the correct rule, and I withdraw the suggestion.

Point 3 — verified working

`GET /apps` returns 200 for a `jry_live_` key and lists id, name and platform. That closes the loop: an integrator can now discover the id they need to pass. The detail that `apps:read` already existed and gated no route is worth keeping in mind — a scope that grants nothing is indistinguishable from a scope that grants everything until someone tests it.

Keeping writes on `settings:write` is the right call, particularly `regenerate-key`.

The web-push clarification is useful

"Neither branch runs, so no mobile job is created" — and such a campaign still sends web push if the workspace has a web app. That explains a class of report you will keep getting: a push campaign that *works*, for a subset of the audience nobody chose, with no indication the mobile half never ran. Our workspace has a web app, so we were one config away from seeing exactly that and concluding push was fine.

One thing still open, from the other report

Android push in our workspace fails at the provider with `FCM requires firebase-admin to be installed`, thrown from a `catch` that also swallows a bad `admin.credential.cert(...)`. That is separate from everything above, and I could not tell from this reply whether it was in the same deploy. Say the word and I will re-run the end-to-end — service account is uploaded, sender id is set, and the device now carries `deviceInfo.appId`.

One more, small

`POST /v1/attributes` with `{"key": null}` does not unset the key. `$unset` does, and it is what the SDK sends — that part is fine. What is not: the response to the `null` write returned a heavily truncated attribute set (2 of 33), which reads exactly like data loss. It was not — the attributes were intact — but I had already written a restore script before I confirmed that. A response that shows a partial profile after a write is a bad thing to be wrong about.